

Universidade Federal de Santa Catarina · Centro Tecnológico
Departamento de Informática e Estatística · Ciência da Computação

Polyglot Import CSV

Uma solução para importação de documentos CSV para múltiplos bancos de dados

Lucas Bueno Cesario
Orientador: Prof. Dr. Ronaldo dos Santos Mello

Roteiro

1 Introdução

2 Fundamentação e trabalhos relacionados

3 Proposta

4 Avaliação

5 Conclusão

Roteiro

1 **Introdução**

2 Fundamentação e trabalhos relacionados

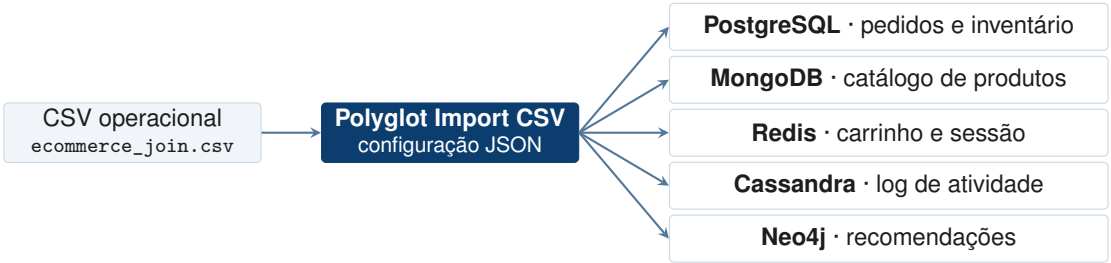
3 Proposta

4 Avaliação

5 Conclusão

Persistência poliglota

- Um só modelo de dados não atende todas as necessidades.
- **Persistência poliglota**: cada tipo de dado fica no SGBD (sistema gerenciador de banco de dados) mais adequado.



Cada dado do e-commerce no SGBD mais adequado. CSV: texto com valores separados por vírgula. JSON: texto estruturado.

O problema

- Cada SGBD tem o seu **importador nativo**, com sintaxe própria.
- Cada importador atende um único SGBD por vez.
- Nenhuma ferramenta encontrada importa o mesmo CSV nos cinco.

Importadores nativos de CSV

	PostgreSQL	Redis	MongoDB	Cassandra	Neo4j
Comando	`COPY`	`redis-cli --pipe`	`mongoimport`	`cqlsh` e `sstableloader`	`neo4j-admin import`
Detalhe	colunas na ordem do CSV	arquivo em protocolo bruto	tipo e cabeçalho por parâmetros	tipos e colunas iguais aos da tabela	CSV separado para nós e relacionamentos

Objetivos

Objetivo geral

Desenvolver a ferramenta *Polyglot Import CSV*, que importa CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

Objetivos específicos

1. **Configuração** em JSON (texto estruturado): filtros, chave primária e nome da coluna.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).

Roteiro

1 Introdução

2 Fundamentação e trabalhos relacionados

3 Proposta

4 Avaliação

5 Conclusão

Cinco paradigmas, cinco SGBDs

Relacional
PostgreSQL

Tabelas ligadas por chaves, com transações consistentes.

Exemplo: pedidos e inventário.

Chave-valor
Redis

Cada chave aponta para um valor.

Exemplo: carrinho e sessão.

Documento
MongoDB

Documentos JSON com campos variáveis.

Exemplo: catálogo de produtos.

Colunar
Cassandra

Linhas com famílias de colunas.

Exemplo: log de atividade.

Grafo
Neo4j

Nós e arestas com propriedades.

Exemplo: recomendação de produtos.

Os quatro últimos são os bancos **NoSQL** (não relacionais).
Um SGBD **multimodelo** reúne vários desses modelos no mesmo sistema.

Trabalhos relacionados

Comparação com os trabalhos relacionados ✓ o Cap. 3 afirma — o Cap. 3 não afirma

Trabalho	Foco	Destinos	Entrada CSV	Mapeamento declarativo validado	Revisão sem gravar
Importadores nativos	importação	um SGBD por vez	✓[1]	—	—
ArangoDB e `arangoimport`	importação	um SGBD multimodelo	✓	—	—
dbcrossbar	migração	PostgreSQL, BigQuery, nuvem	✓	—	—
Hackolade	modelagem visual	dezenas de alvos	—	—	—
Frameworks de acesso[2]	acesso em tempo de execução	vários SGBDs	—	—	—
Polystores[3] (BigDAWG)	consulta federada	fontes heterogêneas	—	—	—

[1] Redis: arquivo em protocolo bruto.
[2] Interface para ler e gravar em vários SGBDs (PEREIRA; GUERRA; ROSA, 2025; SACHDEVA; VASAVA, 2024).
[3] Uma só consulta sobre vários SGBDs (consulta federada).

Trabalhos relacionados

Comparação com os trabalhos relacionados ✓ o Cap. 3 afirma — o Cap. 3 não afirma

Trabalho	Foco	Destinos	Entrada CSV	Mapeamento declarativo validado	Revisão sem gravar
Importadores nativos	importação	um SGBD por vez	✓[1]	—	—
ArangoDB e `arangoimport`	importação	um SGBD multimodelo	✓	—	—
dbcrossbar	migração	PostgreSQL, BigQuery, nuvem	✓	—	—
Hackolade	modelagem visual	dezenas de alvos	—	—	—
Frameworks de acesso[2]	acesso em tempo de execução	vários SGBDs	—	—	—
Polystores[3] (BigDAWG)	consulta federada	fontes heterogêneas	—	—	—
Polyglot Import CSV	**importação**	**cinco SGBDs, cinco paradigmas**	✓	✓ **(JSON Schema)**	✓ **(`--dry-run`)**

[1] Redis: arquivo em protocolo bruto.
[2] Interface para ler e gravar em vários SGBDs (PEREIRA; GUERRA; ROSA, 2025; SACHDEVA; VASAVA, 2024).
[3] Uma só consulta sobre vários SGBDs (consulta federada).

Roteiro

1 Introdução
2 Fundamentação e trabalhos relacionados
3 Proposta
4 Avaliação
5 Conclusão

Visão geral

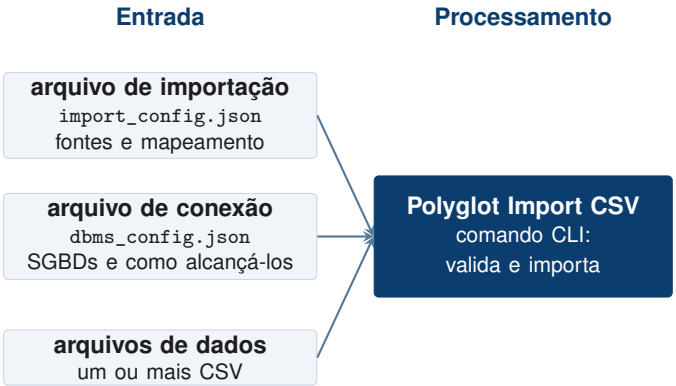
Entrada

arquivo de importação
`import_config.json`
fontes e mapeamento

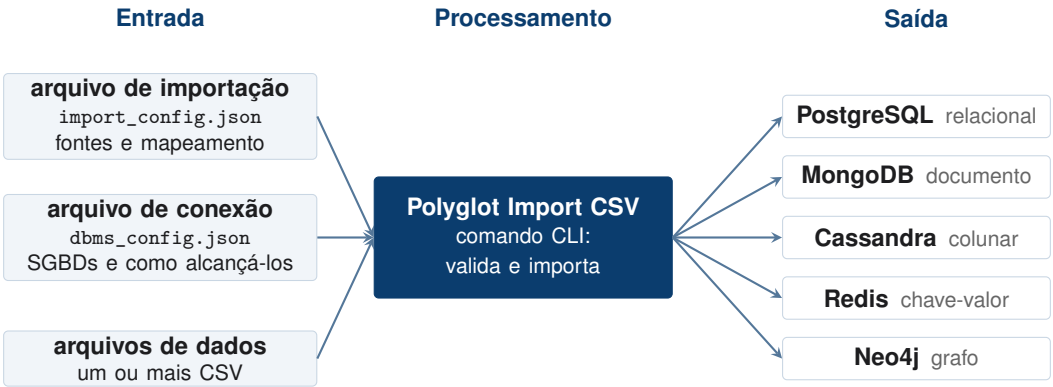
arquivo de conexão
`dbms_config.json`
SGBDs e como alcançá-los

arquivos de dados
um ou mais CSV

Visão geral


Entrada
Processamento
arquivo de importação
import_config.json
fontes e mapeamento
arquivo de conexão
dbms_config.json
SGBDs e como alcançá-los
arquivos de dados
um ou mais CSV
Polyglot Import CSV
comando CLI:
valida e importa

Visão geral



O comando valida as configurações e os dados e só então grava nos SGBDs destino.

Arquivo de importação

Arquivo de importação: JSON com as fontes (CSVs) e o que gravar em cada SGBD.

Trecho de `import_config.json`: a fonte `stock` alimenta a tabela `inventory`.

```
"sources": {
  "stock": "ecommerce_stock.csv",
  "purchase": "ecommerce_purchase.csv", ...
},
"postgres": {
  "entities": { ...
    "inventory": {
      "source": "stock",
      "columns": {
        "product_id": { "is_key": true, "db_type": "BIGINT" },
        "quantity_available": { "db_type": "BIGINT" }, ...
      } }, ...
  }, ...
}, ...
```

- `sources`: dá um nome a cada CSV.
- `postgres`: um bloco por SGBD destino.
- `entities`: tabelas, coleções, nós...
- `columns`: colunas, com chave (`is_key`) e tipo (`db_type`).

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

A validação prévia tem **três etapas**, todas antes de gravar:

arquivo de importação

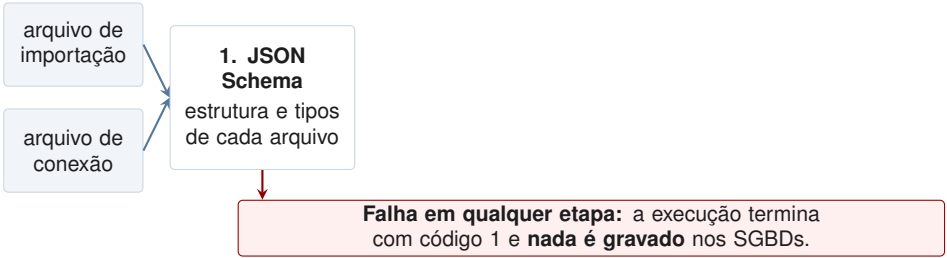
arquivo de conexão

Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

A validação prévia tem **três etapas**, todas antes de gravar:

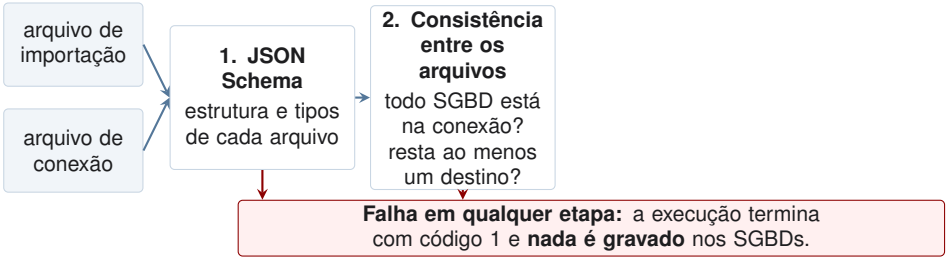


Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

A validação prévia tem **três etapas**, todas antes de gravar:

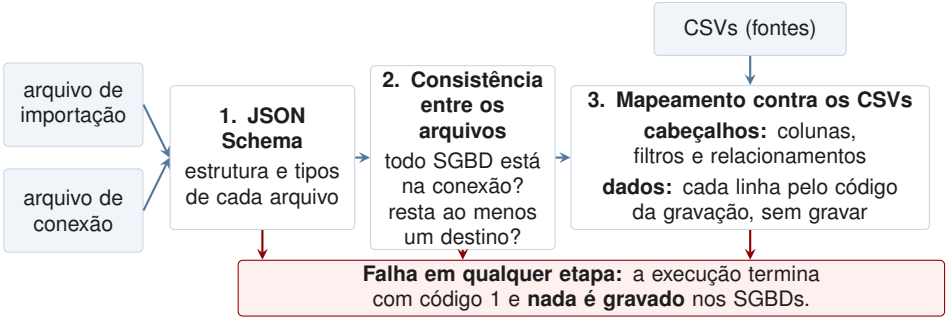


Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

A validação prévia tem **três etapas**, todas antes de gravar:

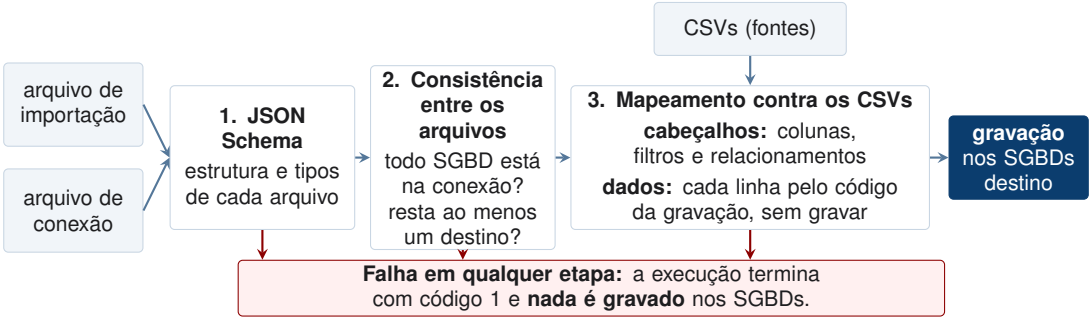


Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Conexão e validação prévia

Arquivo de conexão: JSON que diz onde alcançar cada SGBD.

A validação prévia tem **três etapas**, todas antes de gravar:



Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Cenário exemplo

Tipos de evento do cenário exemplo (e-commerce) todo evento registra o instante e o usuário envolvido

Tipo	Evento	Demais dados registrados
`stock`	reposição de estoque	produto, categoria, quantidade disponível e preço
`purchase`	compra	pedido, produto, contraparte da venda, pagamento e avaliação
`select_product`	seleção de produto	produto selecionado
`add_to_cart`	adição ao carrinho	carrinho, produto e quantidade

Fonte: Tabela 1 do relatório.

Cenário exemplo

Tipos de evento do cenário exemplo (e-commerce) todo evento registra o instante e o usuário envolvido

Tipo	Evento	Demais dados registrados
`stock`	reposição de estoque	produto, categoria, quantidade disponível e preço
`purchase`	compra	pedido, produto, contraparte da venda, pagamento e avaliação
`select_product`	seleção de produto	produto selecionado
`add_to_cart`	adição ao carrinho	carrinho, produto e quantidade

Fonte: Tabela 1 do relatório.

Multifonte um CSV por tipo de evento

```
"sources": {
  "stock": "ecommerce_stock.csv",
  "purchase": "ecommerce_purchase.csv", ...
}
```

Combinado um CSV com coluna de origem

```
"sources": {
  "ecommerce": {
    "file": "ecommerce_join.csv",
    "origin_column": true } }
```

Nos dois modos, o resto do arquivo de importação é igual. O exemplo usa o multifonte.

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory

product_id	2
quantity_available	2
last_restock_date	02/11/2023 03:30
price	50.50

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory

product_id	2
quantity_available	2
last_restock_date	02/11/2023 03:30
price	50.50

MongoDB
documento

← stock
product_catalog

```
{"product_id": 2,
 ...,
 "category": {...},
 "stock": {
  "quantity_available":
   2, ...}}
```

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.50

MongoDB
documento

← stock
product_catalog
{"product_id": 2,
...,
"category": {...},
"stock": {
"quantity_available":
2, ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time
event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.50

MongoDB
documento

← stock
product_catalog
{"product_id": 2,
...,
"category": {...},
"stock": {
"quantity_available":
2, ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

Redis
chave-valor

← add_to_cart
shopping_cart
chave user:7fd7...:cart
valor {...,
"cart_product_id": 3,
"cart_quantity": 1}

← select_product
user_session
chave 7fd7...
valor {"user_name":
"user1", ...}

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.50

MongoDB
documento

← stock
product_catalog
{"product_id": 2,
...,
"category": {...},
"stock": {
"quantity_available":
2, ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

Redis
chave-valor

← add_to_cart
shopping_cart
chave user:7fd7...:cart
valor {...,
"cart_product_id": 3,
"cart_quantity": 1}

← select_product
user_session
chave 7fd7...
valor {"user_name":
"user1", ...}

Neo4j
grafo

← stock e purchase
(:User {...,
user_name: "user1",
...})
-[:PURCHASED {
order_number:
"ORD1", ...}]->
(:Product {
product_id: 2, ...})

Fluxo de execução

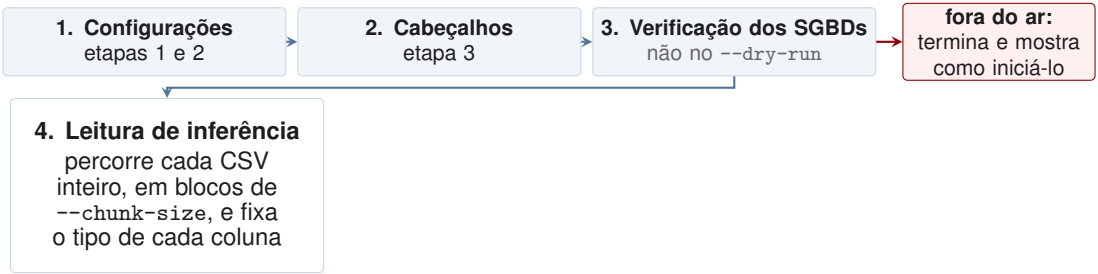
A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Fluxo de execução

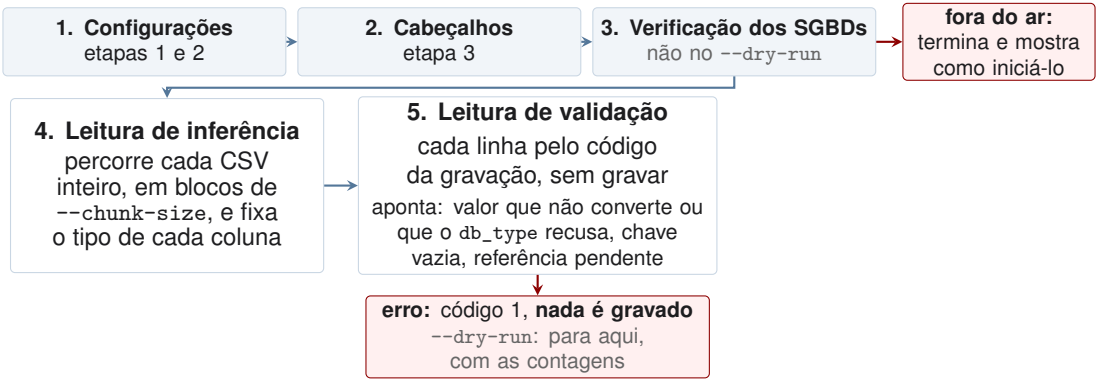
A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Fluxo de execução

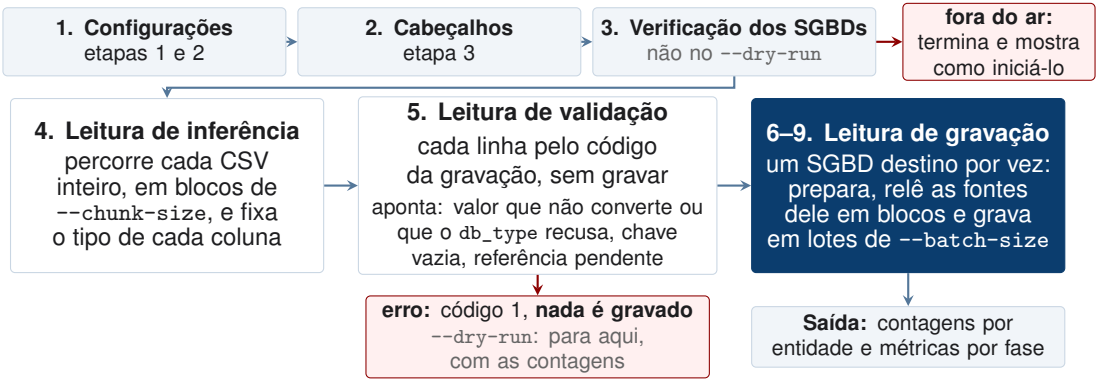
A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Fluxo de execução

A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Exemplo de execução

```
python -m polyglotimportcsv \                          ← executa a ferramenta
  --config data/ecommerce/import_config.json \         ← arquivo de importação
  --dbms-config data/ecommerce/dbms_config.json \      ← arquivo de conexão
  --dry-run                                            ← valida e conta, sem gravar
```

Exemplo de execução

```
python -m polyglotimportcsv \                          ← executa a ferramenta
  --config data/ecommerce/import_config.json \         ← arquivo de importação
  --dbms-config data/ecommerce/dbms_config.json \      ← arquivo de conexão
  --dry-run                                            ← valida e conta, sem gravar
```

Linhas a gravar, segundo o `--dry-run` — após os filtros, antes de eliminar duplicatas

SGBD	Destino	Linhas
PostgreSQL	`categories`	8
PostgreSQL	`products`	8
PostgreSQL	`inventory`	8
PostgreSQL	`orders`	8
MongoDB	`product_catalog`	8
MongoDB	`orders_state_SP`	2
MongoDB	`orders_state_CA`, RJ, RS, SC, SE e TO	1 cada

SGBD	Destino	Linhas
Cassandra	`user_activity_log`	32 (união das 4 fontes)
Redis	`shopping_cart`	8
Redis	`user_session`	8
Neo4j	nó `User`	8
Neo4j	nó `Product`	8
Neo4j	aresta `PURCHASED`	8 (uma por linha da fonte `purchase`)

Fonte: Tabela 12 do relatório.

Escrita e leitura em blocos

Estratégia de escrita como os valores viram tipos nativos e são gravados

ingênua (*naive*)
TCC1, linha de base histórica

Convertia cada célula e gravava cada linha com um comando próprio.

×

em lote (*batch*)
a única da versão final

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

Escrita e leitura em blocos

Estratégia de escrita como os valores viram tipos nativos e são gravados

ingênua (*naive*)
TCC1, linha de base histórica

Convertia cada célula e gravava cada linha com um comando próprio.

×

em lote (*batch*)
a única da versão final

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

Modelo de execução como os arquivos ocupam a memória

materialização
linha de base histórica

Carregava cada fonte inteira na memória: um arquivo maior que ela não chegaria a ser carregado.

×

leitura em blocos (*streaming*)
o único da versão final

Lê em blocos de `--chunk-size` (8 MB) e grava em lotes de `--batch-size` (1024 linhas). O pico acompanha o bloco.

Escrita e leitura em blocos

Estratégia de escrita como os valores viram tipos nativos e são gravados

ingênua (*naive*)		**em lote (*batch*)**
TCC1, linha de base histórica		a única da versão final
Convertia cada célula e gravava cada linha com um comando próprio.	×	Converte uma coluna inteira de uma vez e grava muitos registros por comando.

Modelo de execução como os arquivos ocupam a memória

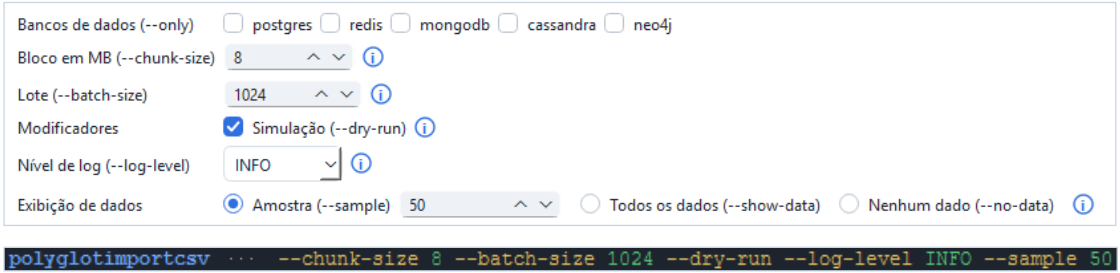
materialização		**leitura em blocos (*streaming*)**
linha de base histórica		o único da versão final
Carregava cada fonte inteira na memória: um arquivo maior que ela não chegaria a ser carregado.	×	Lê em blocos de `--chunk-size` (8 MB) e grava em lotes de `--batch-size` (1024 linhas). O pico acompanha o bloco.

As linhas de base históricas são as que a avaliação compara (QP4 a QP6).

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

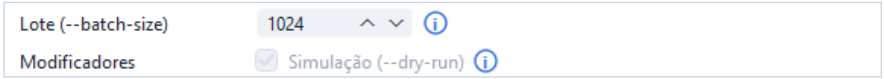


Ocioso: um controle para cada opção de uso cotidiano, com ícone de ajuda, inclusive o bloco e o lote, e o comando montado, colorido e somente leitura.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

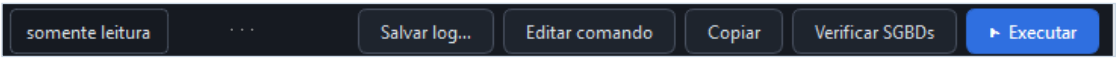


Executando: formulário bloqueado, botão de interrupção e saída colorida da CLI chegando ao console em tempo real.

Interface gráfica

O comando exibido é o comando executado.

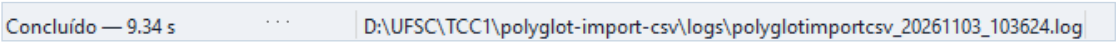
recortes das capturas de tela



Import metrics

dbms	entity	phase	rows	seconds	rows/s
(sources)	*	infer	32	0.560	57
(all)	*	validate	32	0.911	35

✓ Finished dry-run — nothing was written

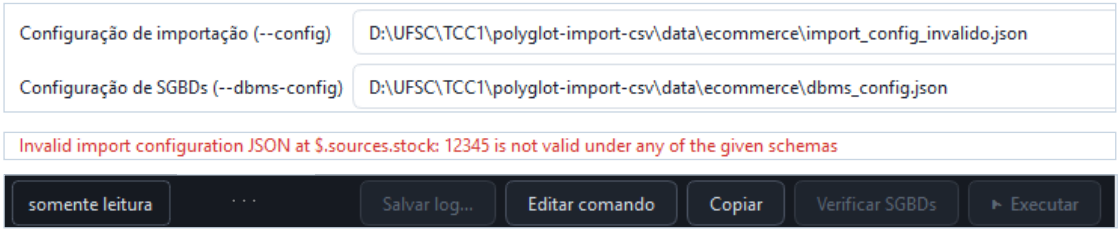


Concluído: métricas por fase, desfecho e arquivo de log da sessão na barra de estado.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela

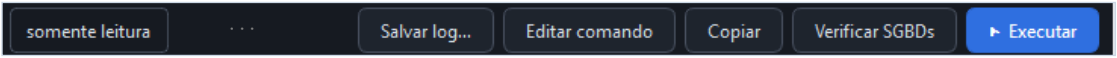


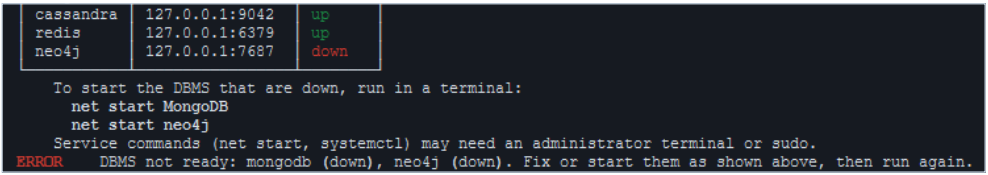
Erro antes de executar: o arquivo de importação viola o *JSON Schema*; a mensagem da CLI cita o campo recusado, e o botão Executar fica desabilitado.

Interface gráfica

O comando exibido é o comando executado.

recortes das capturas de tela





Verificação dos SGBDs: Neo4j e MongoDB fora do ar e, abaixo da tabela de estados, os comandos que os iniciam.

Roteiro

1 Introdução
2 Fundamentação e trabalhos relacionados
3 Proposta
4 Avaliação
5 Conclusão

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala N gera sempre os mesmos arquivos.

Volumes. N = 1.000, 10.000 e 100.000 linhas; bloco de 8 MB e, em 100.000 linhas, também de 64 e 256 MB.

Fonte: §5.1, §5.2 e Tabela 14 do relatório.

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala *N* gera sempre os mesmos arquivos.

Volumes. *N* = 1.000, 10.000 e 100.000 linhas; bloco de 8 MB e, em 100.000 linhas, também de 64 e 256 MB.

Repetições. Três por configuração, com os SGBDs limpos antes; vale a mediana (o valor do meio).

Campanhas pareadas. Medir a memória rastreia cada alocação do Python e distorce os tempos. Tempo: campanha sem essa instrumentação. Pico de memória: campanha com ela.

Precisão. Uma célula medida em duas campanhas: 2,7% no tempo e menos de 0,2% na memória. Nas medições históricas, cerca de 20% no tempo.

Fonte: §5.1, §5.2 e Tabela 14 do relatório.

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala *N* gera sempre os mesmos arquivos.

Volumes. *N* = 1.000, 10.000 e 100.000 linhas; bloco de 8 MB e, em 100.000 linhas, também de 64 e 256 MB.

Repetições. Três por configuração, com os SGBDs limpos antes; vale a mediana (o valor do meio).

Campanhas pareadas. Medir a memória rastreia cada alocação do Python e distorce os tempos. Tempo: campanha sem essa instrumentação. Pico de memória: campanha com ela.

Precisão. Uma célula medida em duas campanhas: 2,7% no tempo e menos de 0,2% na memória. Nas medições históricas, cerca de 20% no tempo.

Fonte: §5.1, §5.2 e Tabela 14 do relatório.

Ambiente: uma máquina

Windows 11, 8 GB de memória

Em Docker: PostgreSQL 16, Redis 7, MongoDB 7, Neo4j 5 e Cassandra 4.1, este com 1 GB de memória Java (*heap*).

Metodologia da avaliação

Dados sintéticos. Gerador determinístico: a mesma escala *N* gera sempre os mesmos arquivos.

Volumes. *N* = 1.000, 10.000 e 100.000 linhas; bloco de 8 MB e, em 100.000 linhas, também de 64 e 256 MB.

Repetições. Três por configuração, com os SGBDs limpos antes; vale a mediana (o valor do meio).

Campanhas pareadas. Medir a memória rastreia cada alocação do Python e distorce os tempos. Tempo: campanha sem essa instrumentação. Pico de memória: campanha com ela.

Precisão. Uma célula medida em duas campanhas: 2,7% no tempo e menos de 0,2% na memória. Nas medições históricas, cerca de 20% no tempo.

Fonte: §5.1, §5.2 e Tabela 14 do relatório.

Ambiente: uma máquina

Windows 11, 8 GB de memória

Em Docker: PostgreSQL 16, Redis 7, MongoDB 7, Neo4j 5 e Cassandra 4.1, este com 1 GB de memória Java (*heap*).

Questões de pesquisa (QP)

QP1 Custo por fase e por SGBD
QP2 Modos de importação
QP3 Escalabilidade
QP7 Validação prévia e bloco

QP4 a QP6 históricas: ingênua e materialização

A escrita domina o tempo (QP1)

Escrita: 78,7% do tempo



Inferência 8,1%

Validação 13,2%

Escrita 78,7%
164,84 de 209,51 s

Tempo por fase, em 100 mil linhas.

Fonte: Tabelas 15 e 16 e Figura 18 do relatório; *N* = 100.000, multifonte, bloco de 8 MB; campanha sem instrumentação de memória.

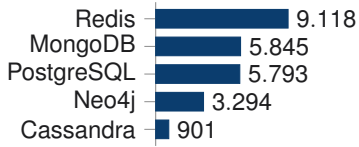
A escrita domina o tempo (QP1)

Escrita: 78,7% do tempo



Tempo por fase, em 100 mil linhas.

Vazão: uma ordem de grandeza



Vazão de escrita: linhas processadas por segundo.

Fonte: Tabelas 15 e 16 e Figura 18 do relatório; *N* = 100.000, multifonte, bloco de 8 MB; campanha sem instrumentação de memória.

A escrita domina o tempo (QP1)

Escrita: 78,7% do tempo



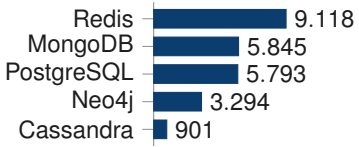
Inferência 8,1%

Validação 13,2%

Escrita 78,7%
164,84 de 209,51 s

Tempo por fase, em 100 mil linhas.

Vazão: uma ordem de grandeza



Vazão de escrita: linhas processadas por segundo.

O Cassandra pesa mais

53%

da importação inteira

110,96 s dos 164,84 s da escrita (67%): grava os eventos das quatro fontes, com 1 GB de *heap*.

Fonte: Tabelas 15 e 16 e Figura 18 do relatório; *N* = 100.000, multifonte, bloco de 8 MB; campanha sem instrumentação de memória.

O tempo cresce com o volume (QP2, QP3)

Tempo total e pico de memória por volume

	Multifonte		Combinado	
Linhas	**Tempo (s)**	**Pico (MB)**	**Tempo (s)**	**Pico (MB)**
1.000	6,51	10,4	5,81	10,2
10.000	22,67	40,1	27,18	44,2
100.000	209,51	66,1	237,39	54,6

Tempos: campanha sem instrumentação de memória. Picos: campanha com instrumentação.

Fonte: Tabelas 17 e 18 do relatório; bloco de 8 MB.

O tempo cresce com o volume (QP2, QP3)

Tempo total e pico de memória por volume

	Multifonte		Combinado	
Linhas	**Tempo (s)**	**Pico (MB)**	**Tempo (s)**	**Pico (MB)**
1.000	6,51	10,4	5,81	10,2
10.000	22,67	40,1	27,18	44,2
100.000	209,51	66,1	237,39	54,6

Tempos: campanha sem instrumentação de memória. Picos: campanha com instrumentação.

Fonte: Tabelas 17 e 18 do relatório; bloco de 8 MB.

Tempo (QP3): quase proporcional a partir de 10.000 linhas: 9,2× e 8,7× para 10× mais dados.

Memória (QP3): cresce bem menos que os dados: 6,4× e 5,3× para 100× mais linhas.

O tempo cresce com o volume (QP2, QP3)

Tempo total e pico de memória por volume

	Multifonte		Combinado	
Linhas	**Tempo (s)**	**Pico (MB)**	**Tempo (s)**	**Pico (MB)**
1.000	6,51	10,4	5,81	10,2
10.000	22,67	40,1	27,18	44,2
100.000	209,51	66,1	237,39	54,6

Tempos: campanha sem instrumentação de memória. Picos: campanha com instrumentação.

Tempo (QP3): quase proporcional a partir de 10.000 linhas: 9,2× e 8,7× para 10× mais dados.

Memória (QP3): cresce bem menos que os dados: 6,4× e 5,3× para 100× mais linhas.

Combinado (QP2): 13% mais tempo e pico menor (54,6 contra 66,1 MB).

O arquivo combinado é 20% maior, tem 45% de células vazias, e cada SGBD relê o arquivo inteiro na gravação.

Fonte: Tabelas 17 e 18 do relatório; bloco de 8 MB.

Validação prévia: um quinto do tempo (QP7)

Inferência e validação, em % do tempo total

Linhas	Multifonte	Combinado
1.000	39,3%	38,5%
10.000	23,4%	25,9%
100.000	21,3%	23,7%

Em troca: nenhum problema detectado chega a uma gravação parcial.

Fonte: Tabelas 19 e 13. Campanhas do bloco: em 8 MB, 204,00 s contra 209,51 s no slide 19 (2,7%).

Validação prévia: um quinto do tempo (QP7)

Inferência e validação, em % do tempo total

Linhas	Multifonte	Combinado
1.000	39,3%	38,5%
10.000	23,4%	25,9%
100.000	21,3%	23,7%

Em troca: nenhum problema detectado chega a uma gravação parcial.

Tamanho do bloco, em 100.000 linhas (multifonte)

Bloco	Pico (MB)	Total	Inferência	Validação
8 MB	66,2	204,00	15,69	27,53
64 MB	131,9	192,91	10,95	22,71
256 MB	149,9	190,88	9,74	23,57

Tempos em segundos.

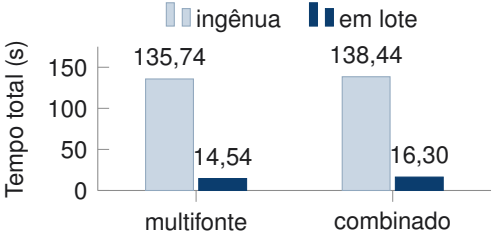
Blocos maiores: $-5{,}4\%$ e $-6{,}4\%$ de tempo, com o dobro ou mais de memória. O padrão de 8 MB favorece a memória.

Fonte: Tabelas 19 e 13. Campanhas do bloco: em 8 MB, 204,00 s contra 209,51 s no slide 19 (2,7%).

Resultados históricos (QP4, QP5, QP6)

Medições com versões anteriores da ferramenta, que ainda ofereciam a escrita ingênua e a materialização.

Em lote: cerca de 9× mais rápida

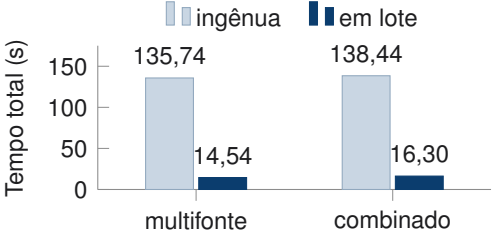


Tempo por estratégia de escrita (N = 10.000).
Fonte: Tabela 20 e Figura 19; sem instrumentação.

Resultados históricos (QP4, QP5, QP6)

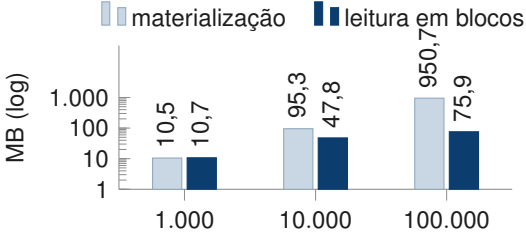
Medições com versões anteriores da ferramenta, que ainda ofereciam a escrita ingênua e a materialização.

Em lote: cerca de 9× mais rápida



Tempo por estratégia de escrita (N = 10.000).
Fonte: Tabela 20 e Figura 19; sem instrumentação.

Leitura em blocos: 12,5× menos memória



Pico de memória por volume. Custo: +10% de tempo em 100.000 linhas (+61% em mil).
Fonte: Tabela 21 e Figura 20 (pico, com instrumentação); Tabela 22 (tempo, sem).

Roteiro

1 Introdução
2 Fundamentação e trabalhos relacionados
3 Proposta
4 Avaliação
5 Conclusão

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Sete questões de pesquisa (QP1–QP7), três delas históricas; de 1.000 a 100.000 linhas; os cinco SGBDs; tempo e memória em campanhas separadas.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Sete questões de pesquisa (QP1–QP7), três delas históricas; de 1.000 a 100.000 linhas; os cinco SGBDs; tempo e memória em campanhas separadas.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).	CLI com `--dry-run`, `--only` e `--chunk-size`; GUI que monta e executa o comando, mantendo-o visível.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Sete questões de pesquisa (QP1–QP7), três delas históricas; de 1.000 a 100.000 linhas; os cinco SGBDs; tempo e memória em campanhas separadas.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).	CLI com `--dry-run`, `--only` e `--chunk-size`; GUI que monta e executa o comando, mantendo-o visível.

Uma ferramenta importa o mesmo CSV para cinco SGBDs de paradigmas diferentes.

Resolve × em aberto

O artigo trata de **evolução de esquemas**; este trabalho, de **importação**.
Três questões do artigo, levadas para a importação (ARAUJO; SILVA; MELLO, 2025):

Questão do artigo	Na importação, com a Polyglot Import CSV	Situação
Gerar instruções adequadas a cada SGBD	Um só arquivo de importação gera a escrita de cada SGBD: SQL, CQL (Cassandra), Cypher (Neo4j), comandos do Redis e documentos do MongoDB.	**resolve**
Estimar o efeito das modificações antes que sejam executadas	Validação prévia e `--dry-run`: contagens por SGBD e por entidade, sem gravar.	**resolve**

Resolve × em aberto

O artigo trata de **evolução de esquemas**; este trabalho, de **importação**.
Três questões do artigo, levadas para a importação (ARAUJO; SILVA; MELLO, 2025):

Questão do artigo	Na importação, com a Polyglot Import CSV	Situação
Gerar instruções adequadas a cada SGBD	Um só arquivo de importação gera a escrita de cada SGBD: SQL, CQL (Cassandra), Cypher (Neo4j), comandos do Redis e documentos do MongoDB.	**resolve**
Estimar o efeito das modificações antes que sejam executadas	Validação prévia e `--dry-run`: contagens por SGBD e por entidade, sem gravar.	**resolve**
Garantir atomicidade (tudo ou nada)	A validação prévia barra os erros de configuração e de dados, mas a gravação vai de um SGBD por vez, sem transação comum: uma falha durante ela, como a queda de um SGBD, deixa uma carga parcial.	**em aberto**

Trabalhos futuros

Um sexto paradigma

Relacional	Chave-valor	Documento	Colunar	Grafo	Motor de busca
PostgreSQL	Redis	MongoDB	Cassandra	Neo4j	Elasticsearch

O Elasticsearch entraria como um novo destino no mesmo núcleo de importação.

Outras frentes

- **Edição da configuração na GUI**, com validação durante a digitação.
- **Milhões de linhas**, com SGBDs dedicados e ajustados.
- **Coerção configurável**: o usuário define como tratar valores malformados.
- **Outros formatos de entrada**: TSV e planilhas Excel.



Universidade Federal de Santa Catarina · Centro Tecnológico
Departamento de Informática e Estatística · Ciência da Computação

Obrigado! Perguntas?

Lucas Bueno Cesario

Falha no meio da importação

- **Antes de gravar**, as três etapas de validação conferem configuração e dados, e todos os SGBDs destino precisam responder. Qualquer falha: nada é gravado.
- **Um SGBD por vez**, em lotes: PostgreSQL, MongoDB, Cassandra, Redis, Neo4j.
- **Sem atomicidade entre SGBDs** (em aberto, slide 23): não há transação comum nem desfazer (*rollback*).
- **Falha durante a gravação**, como a queda de um SGBD: os anteriores ficam gravados, o atual fica parcial, os seguintes ficam vazios; a saída lista os já gravados.
- **Reexecução** com `--only`: com chave, PostgreSQL, Cassandra, Redis e Neo4j não duplicam. No MongoDB, sem `_id` mapeado, duplica documentos; com `_id`, para na chave repetida.

Coerção de tipos

O CSV é lido como **texto**. A **leitura de inferência** percorre o arquivo inteiro e classifica cada coluna, nesta ordem:

1. Booleano
todos os valores não vazios são `true` ou `false`

→

2. Número
todos são inteiros ou reais

→

3. Data e hora
mais de 85% dos não vazios são datas

→

4. Texto
nenhum dos casos anteriores

- Célula vazia vira nulo em número, booleano e data; em texto, fica vazia.
- Valor só com dígitos nunca conta como data: uma coluna de IDs não vira data.
- A **leitura de validação** aponta, com arquivo, linha, coluna e valor, o valor que não converte para o tipo da coluna e o que o tipo declarado (`db_type`) recusaria. Nada é gravado.
- No PostgreSQL, chave repetida não duplica; chave vazia é apontada na validação.

Política de coerção configurável: trabalho futuro (Cap. 6 do relatório).

Por que configurar à mão?

Mapeamento declarativo
o que a ferramenta faz hoje

- Quem projeta decide que dado vai para cada SGBD, com chaves, partições e arestas.
- O arquivo pode ser revisto, e o `--dry-run` o valida sem gravar.
- A mesma configuração reproduz a mesma carga.
- Parte já é automática: sem `columns`, entram todas as colunas, com tipos inferidos.

Mapeamento inferido
por um modelo de linguagem (LLM)

- Poupa a escrita inicial do arquivo.
- O CSV não diz como a aplicação consulta os dados. É isso que decide o SGBD de cada dado.
- A proposta ainda precisa de revisão.

Complementares: um LLM poderia propor o arquivo de importação, e o `--dry-run` o conferiria sem gravar. Caminho possível, fora do escopo deste trabalho.

Importadores nativos: o que cada um exige

SGBD	Importador	O que exige
PostgreSQL	`COPY`	colunas na ordem do CSV; arquivo na máquina do servidor; permissão de superusuário
Redis	`redis-cli --pipe`	arquivo prévio com os comandos (`SET chave valor`); resumo só com os totais de erros e respostas
MongoDB	`mongoimport`	banco, coleção e formato por parâmetros; uma coleção por comando
Cassandra	`cqlsh`, `sstableloader`	tipos e colunas iguais aos da tabela; o `sstableloader` exige SSTables (formato próprio)
Neo4j	`neo4j-admin import`	CSVs separados para nós e para relacionamentos; banco novo, vazio
Polyglot Import CSV		**o mesmo CSV nos cinco SGBDs: um arquivo de importação, validação prévia e `--dry-run`**

Comparação qualitativa: o Cap. 5 não mede esses importadores. Fonte: §3.1 do relatório.

Cassandra e o ambiente

Escrita por SGBD — linhas processadas

SGBD	Linhas	Tempo (s)	Linhas/s
Cassandra	100.000	**110,96**	**901**
Neo4j	89.275	27,10	3.294
PostgreSQL	73.660	12,72	5.793
MongoDB	50.442	8,63	5.845
Redis	49.558	5,44	9.118

O Cassandra processa mais linhas e mais devagar: **110,96 s dos 164,84 s** da escrita, cerca de 67%.

Fonte: Tabela 16 do relatório; N = 100.000, multifonte, bloco de 8 MB; sem instrumentação de memória.

Ambiente (§5.2)

Uma máquina, 8 GB de memória, os cinco SGBDs em contêineres Docker. Cassandra: um nó, *heap* de 1 GB numa máquina virtual Docker de 3,88 GiB.

Leitura do relatório (§5.4.1)

As 901 linhas/s refletem o *heap* pequeno: restrição do ambiente, não do SGBD.

Escala: trabalho futuro

SGBDs dedicados e ajustados; escrita do Cassandra além da concorrência atual.

Ameaças à validade

De construção
mede o que se quer?

- Tempo: 2,7% entre campanhas, numa só célula (históricas: cerca de 20%).
- Memória estável: menos de 0,2%.
- Tempo e memória de campanhas separadas; cargas diferentes nas históricas.

Interna
o efeito vem do fator?

- Gerador determinístico e SGBDs limpos antes de cada repetição.
- Volume moderado: até 100.000 linhas; o bloco de 256 MB já lê cada fonte inteira.
- Sem caches quentes nem execuções concorrentes.

Externa
vale fora deste ambiente?

- Uma máquina, Windows, cinco SGBDs em contêineres com 8 GB: não é produção.
- Cassandra limitado pelo *heap* de 1 GB.
- Dados sintéticos de um só domínio, o e-commerce (§5.1.1).

Fonte: §5.4.1 do relatório.

Referências

ARAUJO, L. N.; SILVA, H. A. B. d.; MELLO, R. d. S. Uma revisão sistemática simplificada e análise de estudos sobre evolução de esquemas em aplicações de persistência poliglota. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2025. DOI: 10.5753/erbd.2025.7322.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. DOI: 10.5220/0013369300003929.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. DOI: 10.1109/ICEEICT61591.2024.10718481.